PulseGuard MLOps report

Coursework report for an educational heart-disease risk scoring service. Repository: https://github.com/2025ae05299/MLOPS_assignment

This document is not medical advice. The model does not diagnose heart disease and must not be used to make a clinical decision.

1. Objective

The assignment asks for a reproducible classifier, experiment tracking, automated tests, a containerized prediction API, a deployment manifest, and basic monitoring. PulseGuard answers that with one design choice that is easy to miss in a copied Cleveland notebook: it trains on all four processed UCI sites and treats site shift as part of the problem.

The served artifact is a scikit-learn Pipeline. Cleaning, sentinel handling, engineered fields, imputation, encoding, and the selected estimator travel together. The API never reimplements those steps.

2. Setup

From a clean checkout:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install -e .
python scripts/download_data.py
python scripts/run_eda.py
python scripts/train.py
pytest
python scripts/infer.py examples/patient_elevated.json
```

`Makefile` wraps the same commands. Python 3.9 is pinned in CI and in the serving image because the committed joblib bundle is produced on 3.9. Serving dependencies are split into `requirements-serve.txt` so the image does not carry MLflow, Matplotlib, or Pytest.

Raw files are downloaded from `https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/`. The script records a SHA-256 for each file in `data/processed/manifest.json`. If the raw files are already present, the script does not download them again.

3. Data acquisition

Citation: Janosi, Andras, Steinbrunn, William, Pfisterer, Matthias, and Detrano, Robert. Heart Disease. UCI Machine Learning Repository, 1988. https://doi.org/10.24432/C52P4X.

The processed files are Cleveland (303), Hungarian (294), Switzerland (123), and VA Long Beach (200). After dropping rows with no age or no severity label, the modeling table has 920 rows and a positive rate of 0.553.

The binary target `heart_disease` is 1 when the original UCI severity field `num` is greater than 0. Severity itself is retained for EDA and is not a model input. This is a label simplification for coursework. It is not a diagnosis.

4. Exploratory analysis

The important data-quality finding is not a histogram of age. Switzerland stores unmeasured cholesterol as 0. A zero is not a lipid measurement. The cleaner converts cholesterol, resting blood pressure, and maximum heart rate sentinels to missing before charts and models see them. Question marks in the UCI text files are also missing.

Missing rates after that cleanup:

- trestbps: 0.065
- chol: 0.220
- fbs: 0.098
- restecg: 0.002
- thalach: 0.060
- exang: 0.060
- oldpeak: 0.067
- slope: 0.336
- ca: 0.664
- thal: 0.528

`ca` and `thal` are populated far more often in Cleveland than in the other sites. Mode-imputing them would paint the Cleveland mode onto Switzerland and VA. The pipeline keeps an explicit missing category and a missingness flag instead.

Class balance is plotted by site because the sites do not share the same prevalence. A model that ignores site can look strong on a shuffled split and still fail when the next batch comes from one hospital. `source_site` is therefore an input, and held-out ROC-AUC is also reported per site.

Figures:

- `reports/figures/class_balance.png`

- `reports/figures/histograms.png`
- `reports/figures/correlation_heatmap.png`
- `reports/figures/missingness_by_site.png`
- `reports/figures/cholesterol_by_target.png`

5. Feature engineering

The API accepts only the 13 published clinical fields plus optional `source_site`. Everything else is created inside `PulseGuardFeatureBuilder`, which is the first step of the saved pipeline.

Constructed fields:

- `st_burden`: exercise ST depression times ST slope. This is a compact stand-in for how large and how shaped the ST change is.
- `pressure_age_index`: resting blood pressure times age, scaled by 100. It lets a linear model see a simple age-pressure interaction without a manual product column at the boundary.
- `hr_gap`: `(220 - age) - thalach`. This is not a measured heart-rate reserve. It is a rough gap between an age-estimated maximum and the recorded maximum, useful only as a model feature.
- `ischemic_combo`: exercise-induced angina together with oldpeak of at least 1.0.
- `age_band`: under 40, 40-54, 55-64, and 65 or older.
- Missingness indicators for cholesterol, vessel count, thalassemia code, and slope.

Numeric fields are median-imputed and standardized inside the training split only. Categorical codes are one-hot encoded with unknown codes ignored, so a future site label does not crash scoring. Binary flags use most-frequent imputation as a safety net.

6. Model development and selection

Candidates:

1. L2 logistic regression with balanced class weights. This is the interpretable baseline. `C` is searched over 0.2, 1.0, and 4.0.
2. Random forest with balanced class weights. Trees, depth, and leaf size are searched. This is the non-linear tabular baseline.
3. Histogram gradient boosting with balanced class weights. Learning rate and depth are searched. This is the challenger, not an automatic winner.

The seed is 5299, taken from the repository owner id `2025ae05299`, so the split is not the usual seed 42.

The data is split 80/20, stratified, before any search. Grid search and out-of-fold scores use only the training portion. The selection score is:

`utility = 0.70 * ROC-AUC + 0.30 * recall`

Recall uses a temporary 0.5 cutoff during comparison. The weights are an educational screening preference: missing a positive marker is treated as more costly than a false alarm, but AUC still dominates so a model cannot win by labeling every row positive. This is not a hospital policy.

The operating threshold is then chosen on out-of-fold training probabilities by maximizing F-beta with beta 1.5, over cutoffs from 0.20 to 0.80. The holdout split is scored once at that frozen threshold. The served model is the estimator refit on the training portion, not on the holdout rows.

Selected model: `hist_gradient_boosting`.
Decision threshold: 0.210.
High-risk band starts at 0.70.

Candidate results:

- hist_gradient_boosting: utility 0.883, CV ROC-AUC 0.896, CV recall 0.850, CV precision 0.859, CV accuracy 0.840.
- random_forest: utility 0.876, CV ROC-AUC 0.893, CV recall 0.838, CV precision 0.834, CV accuracy 0.818.
- logistic_regression: utility 0.875, CV ROC-AUC 0.897, CV recall 0.823, CV precision 0.844, CV accuracy 0.818.

Held-out metrics for the selected model:

- Accuracy: 0.815
- Precision: 0.770
- Recall: 0.951
- F1: 0.851
- ROC-AUC: 0.911
- PR-AUC: 0.913

Per-site held-out ROC-AUC:

- cleveland: rows 66, ROC-AUC 0.935.
- hungarian: rows 58, ROC-AUC 0.869.
- switzerland: rows 24, ROC-AUC n/a.
- va: rows 36, ROC-AUC 0.827.

A site with too few holdout rows, or with only one class in that slice, is reported without an AUC. That is intentional. A tiny slice should not be dressed up as a performance claim.

7. Experiment tracking

MLflow writes to the local `mlruns/` directory. The experiment name is `pulseguard-multisite`. Each candidate run logs the winning hyperparameters, cross-validated accuracy, precision, recall, ROC-AUC, F1, the screening utility, and a scikit-learn model artifact. A final run logs the selected model name, the tuned threshold, held-out metrics, and the comparison plots.

`mlruns/` is gitignored because it duplicates the model bundle and will be regenerated by CI. The compact evidence checked into git is `reports/experiment_summary.json`, `models/MODEL_CARD.md`, and the figures under `reports/figures/`. CI uploads the full MLflow store as a workflow artifact.

Local UI:

mlflow ui --backend-store-uri mlruns

8. Packaging and reproducibility

`models/pulseguard_pipeline.joblib` contains the fitted pipeline and a model card: name, version `1.0.0`, seed, thresholds, held-out metrics, and the educational disclaimer. `scripts/infer.py` loads that bundle and scores a JSON file without starting the API.

Reproducibility controls:

- Pinned `requirements.txt` and a slimmer `requirements-serve.txt`.
- Editable package install so joblib workers can import the custom transformer.
- One feature builder used by training, tests, batch inference, and FastAPI.
- Raw file hashes in `data/processed/manifest.json`.
- The holdout split is not used to choose the model or the threshold.

9. Tests and CI/CD

Pytest covers sentinel cleaning, the committed table contract, engineered features, the selection formula, request validation, the prediction response, and a degraded health check when the model file is missing. API tests fit a tiny logistic pipeline in a temporary directory. They do not depend on the full training run, so a broken serving contract fails before the expensive job.

`.github/workflows/ci.yml` has two jobs:

1. `quality` installs dependencies, runs Ruff, runs Pytest, retrains, scores a sample JSON file, and uploads reports, the model, and `mlruns/`.
2. `container` downloads that model, builds the Docker image, waits for `/health`, and posts `examples/patient_elevated.json` to `/predict`. If either job fails, the logs name the failing step. The Docker job does not run on a red test job.

A human still needs to attach the GitHub Actions screenshot after the first push. The workflow file itself is in the repository.

10. Containerization

The image is `python:3.9-slim`. It creates an unprivileged user, installs serving requirements, copies `src/` and `models/`, and starts Uvicorn on port 8000. Endpoints:

- `GET /health`
- `GET /model-info`
- `GET /metrics`
- `POST /predict`

`/predict` accepts JSON and returns the educational prediction, the probability of the positive label, confidence as the probability of the chosen side, a risk band, model identity, a request id, and the disclaimer. Invalid ranges, including a zero cholesterol value, are rejected with HTTP 422. A missing model returns HTTP 503 so a probe will not send traffic to an empty container.

`docker-compose.yml` adds Prometheus and Grafana beside the API. That is the local production-shaped stack.

11. Deployment

Kubernetes manifests live in `k8s/`: namespace, config, deployment, LoadBalancer service, ingress, and an HPA. A Helm chart in `deploy/helm/pulseguard` covers the same shape. `k8s/servicemonitor.yaml` is optional and is not applied by kustomize, because it requires a Prometheus Operator CRD that a fresh Minikube cluster may not have.

The image pull policy is `IfNotPresent`. For Docker Desktop or Minikube, build `pulseguard-api:1.0.0` locally, then:

```
kubectl apply -k k8s/
kubectl port-forward -n pulseguard svc/pulseguard-api 8000:80
```

No public cloud project was available while this repository was assembled, so there is no live public URL. Deployment instructions, including what would change for a registry push, are in `docs/deployment.md`. Inventing a URL would be worse than documenting the gap.

12. Monitoring and logging

Request logs are JSON on stdout. A prediction log contains the request id, a short hash of the

payload, the prediction, the risk band, the probability, and the source site. It does not log the raw clinical JSON. That is a deliberate coursework choice: the UCI rows are public, but a production scorer should not write patient measurements to an application log.

Prometheus metrics:

- `pulseguard_http_requests_total`
- `pulseguard_request_latency_seconds`
- `pulseguard_predictions_total`
- `pulseguard_positive_probability`

The Grafana dashboard is provisioned from `monitoring/grafana/dashboards/pulseguard.json`. Local login is `admin` / `pulseguard-local`. Change that password before sharing the stack. Uvicorn is started with one worker because the counters are process-local. A multi-worker deployment would need a Prometheus multiprocess registry or a sidecar scrape per worker.

13. Architecture

UCI raw files -> sentinel cleaner -> train/MLflow selection -> joblib pipeline
GitHub Actions -> Docker API -> Prometheus and Grafana
Kubernetes manifests deploy the same image behind a LoadBalancer or Ingress

The rendered diagram is `reports/figures/architecture.png`.

14. What a human still has to do

- Record the walkthrough video using `docs/video_shot_list.md`.
- Save CI, Docker, Kubernetes, Grafana, and MLflow screenshots into `screenshots/`.
- If a cloud account is provided later, push the image and replace `image: pulseguard-api:1.0.0` with the registry path. Then record the external URL.

15. Limitations

- The label is a binarized angiographic status from a historical public study, not an outcome collected for this project.
- Site shift remains visible. A good overall AUC does not mean every site is equally well scored.
- Several engineered fields are proxies. `hr_gap` in particular should not be described as a clinical measurement.
- The threshold policy is a teaching device. A real screening program would set it from costs, capacity, and a prospective study.
- The container is ready to run. A public endpoint is not, until someone with cloud access deploys it.
- Fairness by sex and site is reported only at the level of per-site AUC. That is a starting point, not a

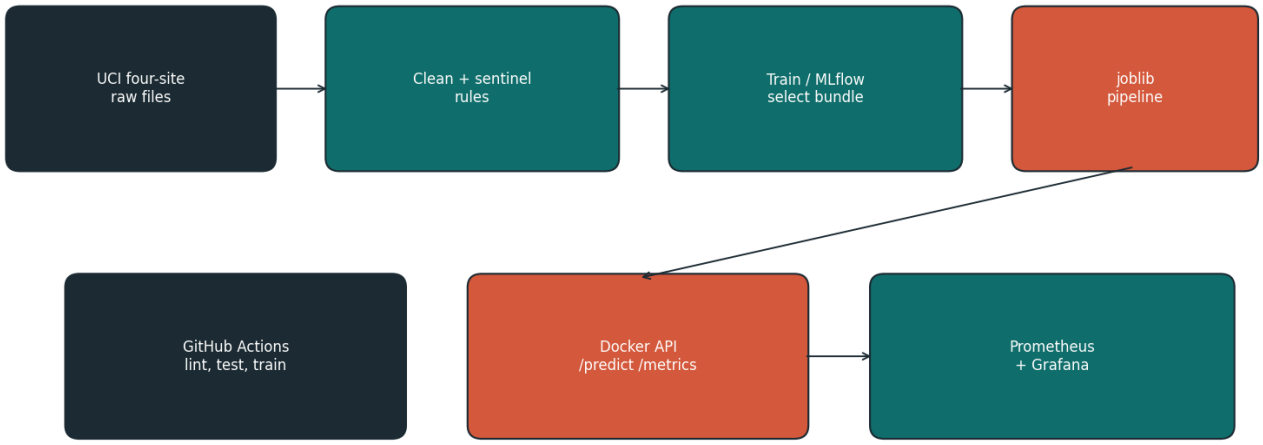
fairness audit.

16. Closing

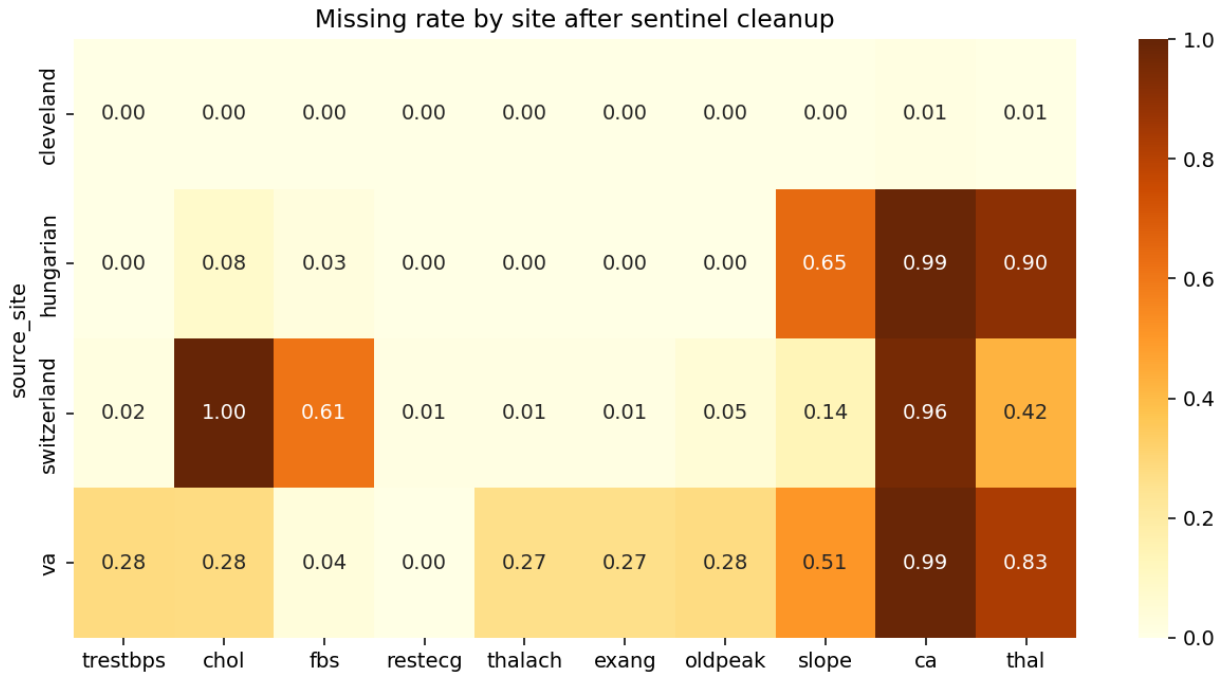
PulseGuard is a small but complete MLOps path: multi-site data with an explicit sentinel rule, two or more compared models, a written selection rule, MLflow runs, a frozen pipeline, tests that fail closed, a CI workflow that trains and probes a container, and a monitoring stack that can be started with Compose. The educational boundary is part of the design. The API says what it is, and the report should be graded as an engineering submission, not as a clinical claim.

architecture.png

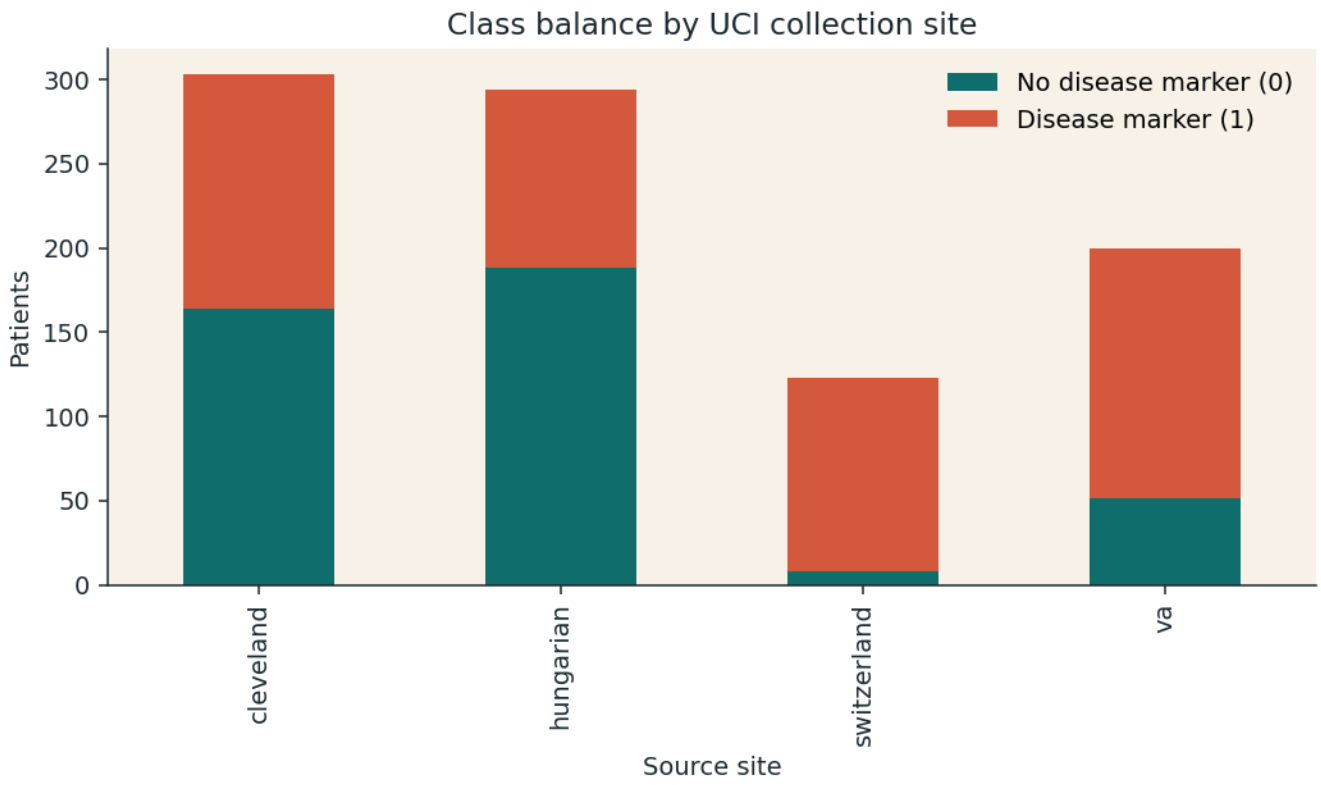
PulseGuard educational architecture



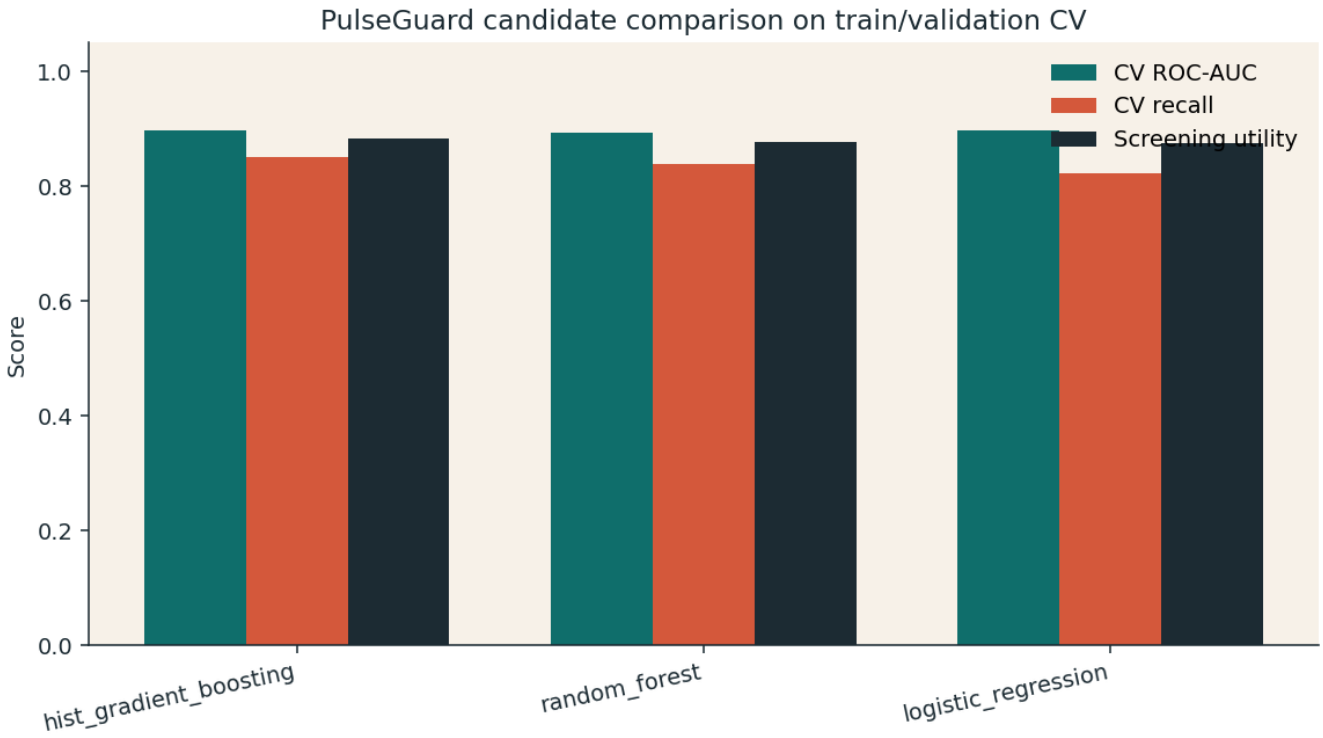
missingness_by_site.png



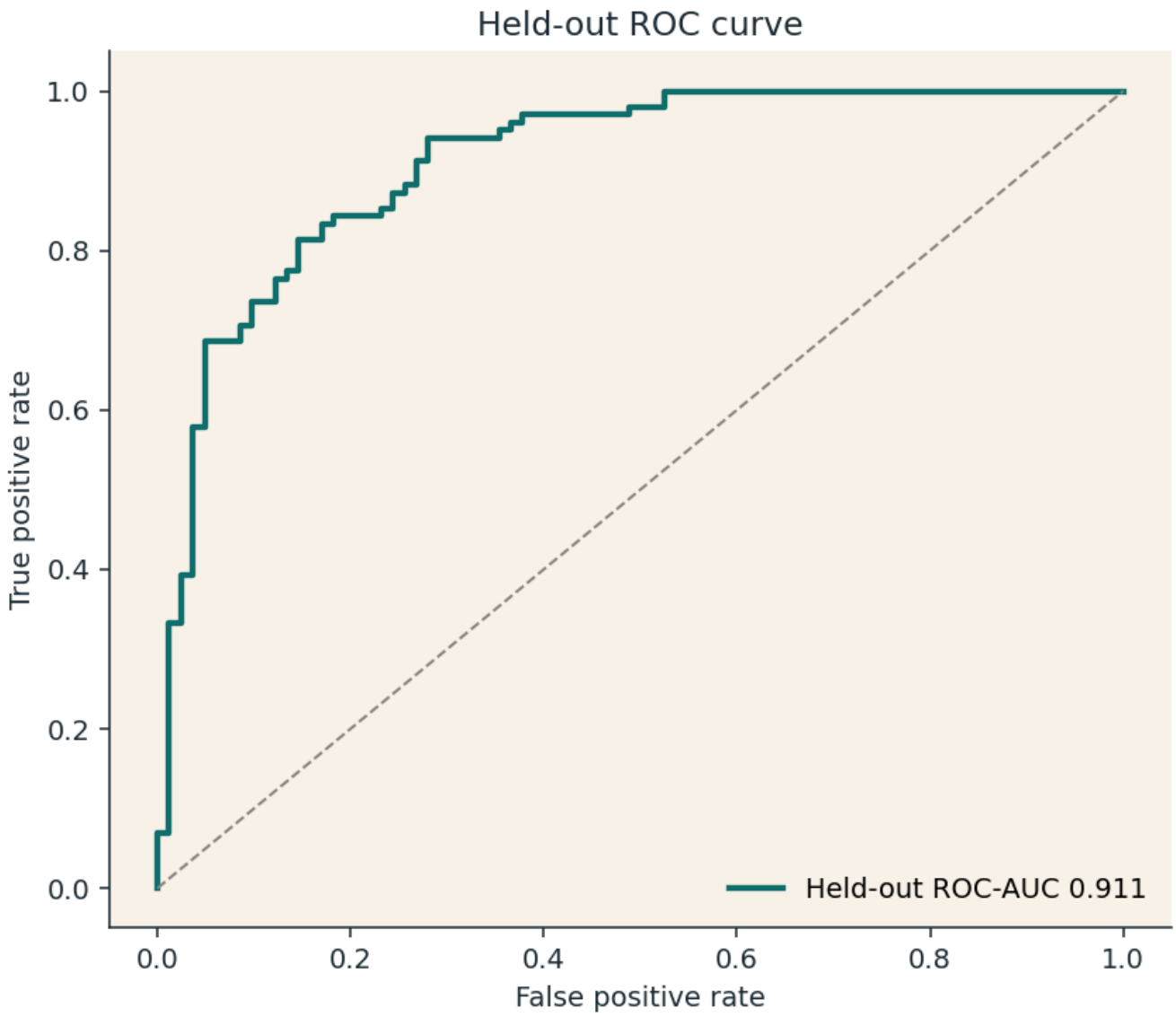
class_balance.png



model_comparison.png



roc_curve.png



confusion_matrix.png

